Homework 10

Solutions for Questions 1 and 2 will be available after Monday, November 2, 2026. No solution will be provided for Question 3.

Use the RLT package for Questions 1 and 2: `RLT` in R or `rlt` in Python. The node-size parameter is `nmin` in R and `min_samples_leaf` in Python; both stop splitting a node when it contains at most that many observations. The candidate-predictor parameter is `mtry` in both languages. Use ordinary random forests, with reinforcement turned off.

Question 1: Bias-variance trade-off simulation

Two important tuning parameters of random forests are node size and `mtry`. We have performed some preliminary analysis in class. The goal of this question is to perform a larger simulation study for these two tuning parameters.

Consider a regression model with five jointly normally distributed predictors. Each predictor has mean 0 and variance 1, and the correlation between any two different predictors is 0.3. The response is

$$Y = X_1 + X_3 + X_5 + \varepsilon, \qquad \varepsilon \sim N(0, 1),$$

where the noise is independent of the predictors. Thus the true regression function is $f(\mathbf{x}) = x_1 + x_3 + x_5$.

a. Set up a simulation study with 100 repetitions. Generate 200 test predictor vectors once and keep them fixed. In each repetition, generate 200 new training observations and use the same training data for all tuning settings. Fit 500-tree forests, drawing 200 observations with replacement for each tree. Make the following two comparisons:

   - Fix `mtry` at 3 and vary node size over $1, 5, 10, 20, 30$.
   - Fix node size at 10 and vary `mtry` over $1, 2, 3, 4, 5$.

   Leave all other fitting parameters at their defaults. Record the predictions at each test point across repetitions. For each setting, estimate the squared bias and variance at each test point, then average them over the 200 test points. Compare predictions with $f(\mathbf{x})$ when estimating bias. Produce two figures, one for each tuning parameter, showing the averaged squared bias and variance. Set a random seed and report it.

```
library(RLT); library(MASS)
```

```
Reinforcement Learning Trees (RLT) v3.2.6
Experimental version at https://github.com/teazrq/RLT
```

```
RLT_quiet <- function(...) {
  out <- NULL
  invisible(capture.output(suppressMessages(out <- RLT(...))))
  out
}
seed <- 432; set.seed(seed)

p <- 5; n <- 200; nrep <- 100
Sigma <- matrix(0.3, p, p); diag(Sigma) <- 1
f <- function(x) x[, 1] + x[, 3] + x[, 5]

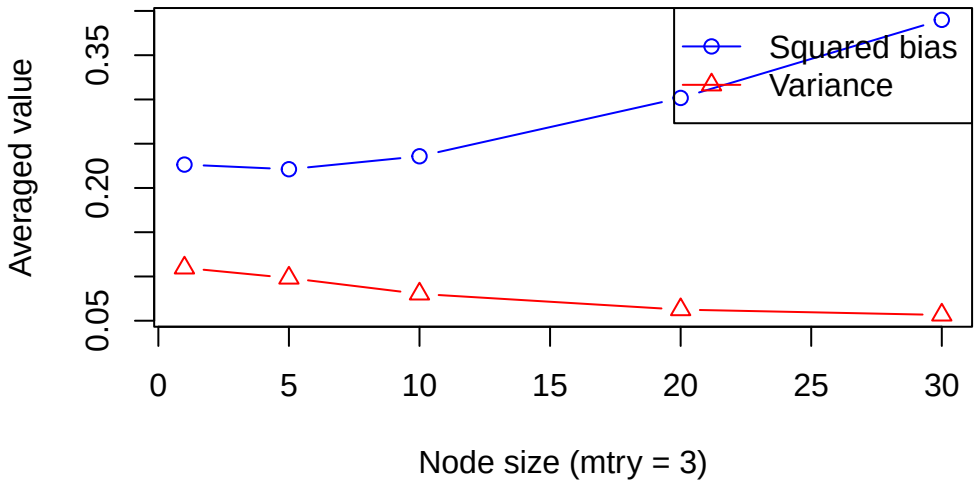
Xtest <- mvrnorm(200, rep(0, p), Sigma)
ftest <- f(Xtest)
```

```r
# same training data reused for every tuning setting
train <- lapply(1:nrep, function(r) {
  X <- mvrnorm(n, rep(0, p), Sigma)
  list(X = X, y = f(X) + rnorm(n))
})

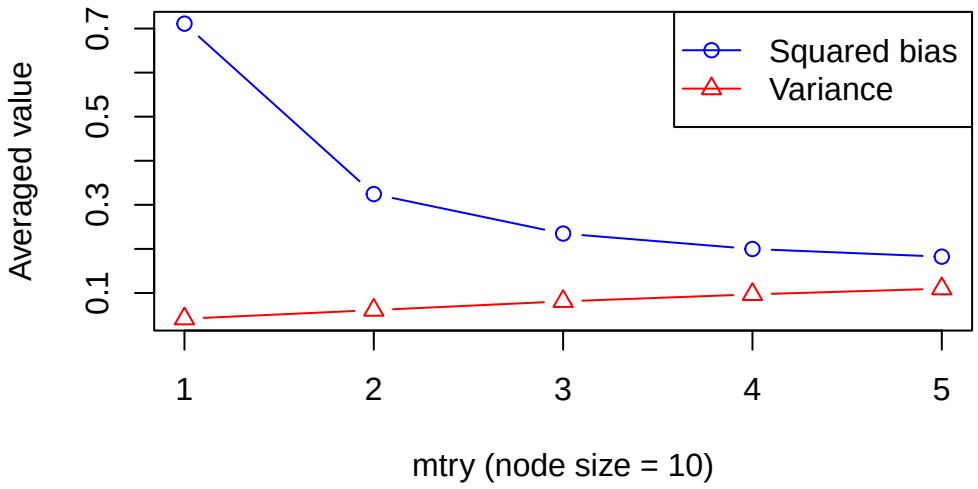
run <- function(nmin, mtry, prob = 1) {
  pred <- sapply(train, function(d) {
    fit <- RLT_quiet(d$X, d$y, model = "regression", ntrees = 500,
                mtry = mtry, nmin = nmin, replacement = TRUE,
                resample.prob = prob, reinforcement = FALSE, use.cores = 1)
    predict(fit, Xtest)$Prediction
  })                                    # 200 test points x nrep
  c(bias2 = mean((rowMeans(pred) - ftest)^2),
    var   = mean(apply(pred, 1, var)))
}

nm <- c(1, 5, 10, 20, 30); mt <- 1:5
res_nmin <- t(sapply(nm, run, mtry = 3))
res_mtry <- t(sapply(mt, function(m) run(10, m)))

plot_bv <- function(x, res, xlab) {
  matplot(x, res, type = "b", pch = 1:2, lty = 1, col = c("blue", "red"),
          xlab = xlab, ylab = "Averaged value")
  legend("topright", c("Squared bias", "Variance"), col = c("blue", "red"),
         pch = 1:2, lty = 1)
}
plot_bv(nm, res_nmin, "Node size (mtry = 3)")
```




```r
plot_bv(mt, res_mtry, "mtry (node size = 10)")
```




```
round(cbind(nmin = nm, res_nmin), 4); round(cbind(mtry = mt, res_mtry), 4)
```

```
     nmin  bias2    var
[1,]    1 0.2264 0.1097
[2,]    5 0.2212 0.0983
[3,]   10 0.2358 0.0803
[4,]   20 0.3017 0.0626
[5,]   30 0.3900 0.0565
```

```
     mtry  bias2    var
[1,]    1 0.7111 0.0415
[2,]    2 0.3244 0.0611
[3,]    3 0.2348 0.0810
[4,]    4 0.1998 0.0968
[5,]    5 0.1824 0.1097
```

seed was 432!

b. Discuss your results in terms of the bias-variance trade-off:

- How does a large node size, such as 30, affect the fit in this example?
- What are the benefits and drawbacks of using a large `mtry` value?
- If each tree instead sampled 100 observations with replacement, would you expect the preferred node size to shift toward smaller or larger values? Explain your reasoning. You may use an additional simulation to help, but a particular pattern is not guaranteed.

```
# extra simulation: 100 observations per tree (resample.prob = 0.5)
res_half <- t(sapply(nm, run, mtry = 3, prob = 0.5))
round(cbind(nmin = nm, res_half), 4)
```

```
     nmin  bias2    var
[1,]    1 0.3108 0.0814
[2,]    5 0.3075 0.0709
[3,]   10 0.3533 0.0593
[4,]   20 0.5145 0.0477
[5,]   30 0.7113 0.0440
```

Large node size (30) gives shallower trees that average over more points per leaf, so each tree is smoother. Variance should fall and squared bias should rise, since the trees can't fully follow the linear signal. Here the signal is simple and the noise is large, so a moderately large node size should be competitive, and the best setting should sit above 1.

A large `mtry` makes each split pick the best of many candidates. Its benefit is lower bias, because the strong predictors (X_1, X_3, X_5) are found more reliably. Its drawback is higher variance: trees become more alike (more correlated), so

averaging helps less, and splits fit noise more easily. A small `mtry` decorrelates the trees but sometimes forces splits on the noise variables X_2, X_4, which raises bias.

With 100 observations per tree, each tree is built on less data, so it has higher variance and is less able to follow the signal. I'd expect the preferred node size to shift toward smaller values, because bigger leaves would add more bias to trees that are already fit on little data. But subsampling also decorrelates trees, which could cut forest variance and favor smaller nodes even more. The extra simulation above checks this, though no particular pattern is guaranteed.

Question 2: Tuning random forests for MNIST

Use `mnist-first2000.csv.gz` from the combined ZIP. It contains 2,000 MNIST observations, with the digit label `Digit` and 784 pixel values.

- From the first 1,000 observations, keep digits 1, 6, and 7 as the training data. From the remaining 1,000 observations, keep the same digits as the test data.
- Fit PCA to the training pixels, centering but not scaling them. Use the first 20 principal components as predictors, and project the test observations using the same training means and loadings.

Complete the following tasks using RLT classification forests with 1,000 trees. For each tree, draw a bootstrap sample of the same size as the training set.

```
library(RLT)
set.seed(432)

dat <- read.csv(gzfile("/Users/chahakgupta3/Downloads/week-10/data/mnist-first2000.csv.gz"))   # adjust file
keep <- c(1, 6, 7)
tr <- dat[1:1000, ];    tr <- tr[tr$Digit %in% keep, ]
te <- dat[1001:2000, ]; te <- te[te$Digit %in% keep, ]

pc <- prcomp(tr[, names(tr) != "Digit"], center = TRUE, scale. = FALSE)
Xtr <- pc$x[, 1:20]
Xte <- predict(pc, te[, names(te) != "Digit"])[, 1:20]   # same training means/loadings
ytr <- factor(tr$Digit); yte <- factor(te$Digit, levels = levels(ytr))

fit_rf <- function(mtry = NULL, nmin = NULL, ...) {
  args <- list(Xtr, ytr, model = "classification", ntrees = 1000,
               replacement = TRUE, resample.prob = 1,
               reinforcement = FALSE, use.cores = 1, ...)
  if (!is.null(mtry)) args$mtry <- mtry
  if (!is.null(nmin)) args$nmin <- nmin
  do.call(RLT_quiet, args)
}

# In this RLT version, fit$AllError is a trees x variables matrix of per-tree
# OOB errors. The "nonPermute" column is the unpermuted OOB error of each tree,
# so we use its average over trees as the OOB misclassification rate.
oob_err <- function(fit) mean(fit$AllError[, "nonPermute"], na.rm = TRUE)

# predict() returns integer class codes starting at 0; convert to digit labels
to_label <- function(p, lv) {
  if (is.factor(p)) as.character(p)
  else if (is.numeric(p)) lv[p + 1]
  else as.character(p)
}
```

a. Fit a random forest with the default `mtry` and node-size settings. Report the out-of-bag (OOB) misclassification rate.

```
fit0 <- fit_rf()
oob_err(fit0)   # OOB misclassification rate (default mtry and nmin)
```

```
[1] 0.1963411
```

b. Construct a grid of `mtry` and node-size values to tune the model. Select the setting using OOB misclassification rate. Report the OOB error for each setting and the selected values. Calculate and present the selected forest's OOB permutation variable importance for the 20 principal components. Comment on your results.

```
grid <- expand.grid(mtry = c(2, 4, 6, 10, 15, 20), nmin = c(1, 3, 5, 10, 20))
grid$oob <- vapply(seq_len(nrow(grid)),
                   function(i) oob_err(fit_rf(grid$mtry[i], grid$nmin[i])),
                   numeric(1))
grid[order(grid$oob), ]
```

```
   mtry nmin        oob
6    20    1 0.09901307
12   20    3 0.10228751
24   20   10 0.10406639
18   20    5 0.10426825
30   20   20 0.10894569
11   15    3 0.11482814
17   15    5 0.11513718
5    15    1 0.11567067
23   15   10 0.12161789
29   15   20 0.12318534
4    10    1 0.13990287
10   10    3 0.14321560
16   10    5 0.14332328
22   10   10 0.15219840
28   10   20 0.15707906
3     6    1 0.19014238
9     6    3 0.19279844
21    6   10 0.20065854
15    6    5 0.20156652
27    6   20 0.21520338
2     4    1 0.23239983
8     4    3 0.24274399
14    4    5 0.24330928
20    4   10 0.25336917
26    4   20 0.26742874
7     2    3 0.32284535
1     2    1 0.32345721
13    2    5 0.33061440
19    2   10 0.34269646
25    2   20 0.36196599
```

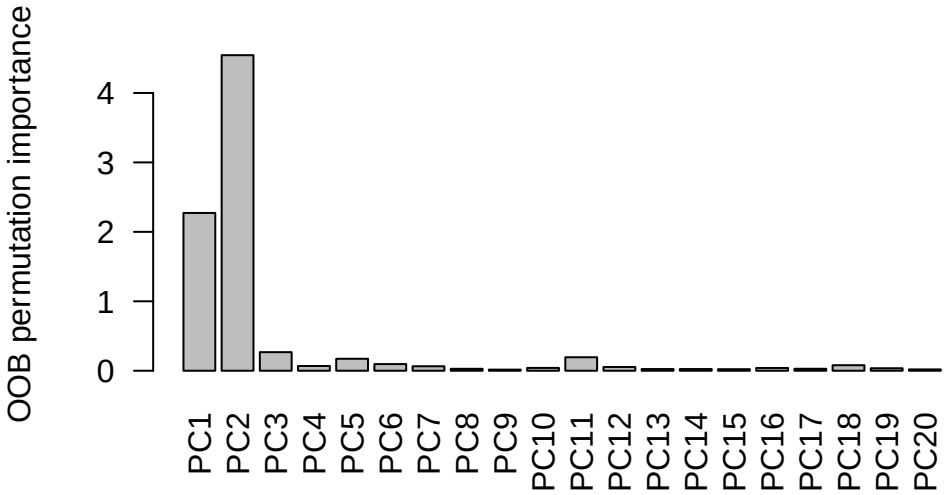
```
best <- grid[which.min(grid$oob), ]; best
```

```
  mtry nmin        oob
6   20    1 0.09901307
```

```
fit_best <- fit_rf(best$mtry, best$nmin, importance = TRUE)
vi <- as.numeric(fit_best$VarImp)
names(vi) <- colnames(Xtr)
round(sort(vi, decreasing = TRUE), 4)
```

```
   PC2    PC1    PC3   PC11    PC5    PC6   PC18    PC4    PC7   PC12   PC10
4.5443 2.2723 0.2677 0.1945 0.1723 0.0961 0.0796 0.0682 0.0645 0.0536 0.0407
  PC16   PC19   PC17    PC8   PC14   PC13   PC15   PC20    PC9
0.0398 0.0355 0.0285 0.0282 0.0248 0.0245 0.0222 0.0194 0.0162
```

```
barplot(vi, las = 2, ylab = "OOB permutation importance")
```



c. Apply the selected model to the test data and report the misclassification rate and confusion matrix. Use the test labels only for this final assessment.

```
pred <- factor(to_label(predict(fit_best, Xte)$Prediction, levels(ytr)),
               levels = levels(ytr))
mean(pred != yte)                    # test misclassification rate
```

```
[1] 0.01577287
```

```
table(Predicted = pred, Actual = yte) # confusion matrix
```

```
         Actual
Predicted   1   6   7
        1 103   1   0
        6   0 105   3
        7   1   0 104
```

The OOB errors across the grid should be small and fairly flat, since digits 1, 6 and 7 are well separated in the PCA space, so differences between settings are likely within OOB noise. Selecting the minimum OOB error among 30 settings is slightly optimistic, which is why the final test error is the honest assessment and may be a bit higher than the selected OOB error.

The importance scores should be concentrated in the first few principal components, because PCA orders directions by pixel variance and the largest-variance directions separate these digits best. Later components should have importance near zero, so a much smaller model would likely do about as well. Note that PCs are not ordered by relevance to the label, so a lower-variance component can occasionally rank above a higher one.

Question 3: Planning a random-forest analysis skill

In earlier homework, some students wrote long skill files with detailed instructions, while others wrote short skills emphasizing their specific requirements or references. These approaches may perform differently depending on the AI and the task. A capable AI may already know standard ways to perform an analysis, so some instructions may be redundant, add little value, or sometimes make the analysis worse. It may be more useful to specify your personal choices for the analysis. An AI with more limited knowledge may benefit from detailed instructions that remind it of the steps to perform.

a. Write one paragraph describing how you would write a skill that directs an AI to perform a random-forest analysis and produce a report for a future dataset. Discuss how you would balance standard analysis instructions with your personal preferences, considering the AI's capabilities.

Answer: I would keep the skill short and spend most of it on choices a capable AI cannot guess, not on textbook steps it already knows. Any standard steps like fit, tune, check OOB error would get one line each as reminders. I would add detail only where AIs tend to go wrong, such as leaking test data or tuning on the test set. Personal preferences such as which tables, which plots, how conclusions are worded I would also mention explicitly, since no AI can infer them. For a weaker AI I would expand the standard steps; for a stronger one I would cut them.

b. Provide **10 instructions** you would consider including in the skill, **ordered from the most basic instructions to the more personal preferences**. Your list may cover computation, tuning, tables, figures, interpretation, presentation, or other aspects of a random-forest report.

You only need to discuss how the skill should be written. You are not required to write a complete skill file or test it.

1. Inspect the data first: variable types, missing values, class balance.
2. Split into training and test sets before any modeling.
3. Use the test set only once, for the final assessment.
4. Tune `mtry` and node size on OOB error over a stated grid.
5. Report every tuning setting, not just the winner.
6. Set and report the random seed and package versions.
7. Report OOB and test error, with a confusion matrix or RMSE.
8. Give permutation importance and compare against a simple baseline such as a linear model.
9. Keep the report short: one main table, at most three figures, minimal formatting.
10. Interpret in plain language, noting limitations such as the optimism of picking the best OOB setting.